PROYECTO FINAL DE COMUNICACIONES II – 27145

Fase 4: Validación final, ajuste y comparación experimental (Guía 4)

MEDICIÓN DE DESEMPEÑO MEDIANTE CURVAS BER vs. Eb/N0

Justificación

En las actividades anteriores cada equipo diseñó un sistema digital Tx–Rx, lo llevó a hardware SDR y demostró una implementación funcional utilizando HackRF One y/o USRP. El siguiente paso consiste en cuantificar el desempeño del enlace. Para ello se incorporará un sistema de instrumentación que permita medir la tasa de error de bit (BER) mientras se realiza un barrido controlado de Eb/N0.

Los archivos suministrados no sustituyen el transmisor ni el receptor desarrollados por el grupo. Su función es únicamente añadir una herramienta común de medición para comparar el comportamiento de los diferentes sistemas bajo un procedimiento reproducible.

Objetivo general

Integrar al sistema SDR ya funcional una herramienta de medición BER que permita obtener automáticamente la curva BER vs. Eb/N0 y analizar cuantitativamente el rendimiento del receptor.

Objetivos específicos

- Identificar correctamente la tasa de bits Rb y la tasa de muestreo de banda base bb_rate del sistema implementado.
- Integrar el bloque e_EbN0_Sweep entre la salida I/Q del SDR receptor y el receptor digital.
- Integrar el bloque e_BER_Curve utilizando como referencia los bits transmitidos y los bits recuperados.
- Alinear temporalmente las secuencias Tx y Rx antes de aceptar una medición BER.
- Obtener automáticamente una curva BER vs. Eb/N0.
- Interpretar la curva obtenida y documentar los parámetros utilizados durante la medición.

Archivos suministrados

Archivo	Función
BER_SDR_INSTRUMENTACION_BASE.grc	Plantilla con variables, Embedded Python Blocks y gráfica de BER.
e_EbN0_Sweep.py	Introduce AWGN controlado y realiza automáticamente el barrido de Eb/N0.
e_BER_Curve.py	Compara bits Tx/Rx y calcula la BER correspondiente a cada punto.
README_BER_SDR.txt	Resumen de parámetros y recomendaciones de integración.

ESTRUCTURA GENERAL DE INTEGRACIÓN

El sistema existente debe conservar su estructura funcional:

Bits Tx → Tx digital → SDR Tx → canal físico → SDR Rx → Rx digital → Bits Rx

La instrumentación BER se incorpora en dos puntos:

SDR Rx → adaptación de tasa → e_EbN0_Sweep → Rx digital

Bits Tx → e_BER_Curve ← Bits Rx

La salida del segundo bloque se conecta a un QT GUI Vector Sink para visualizar:

BER = f(Eb/N0)

IMPORTANTE: La medición debe incorporarse sobre una copia del sistema que ya funciona. No cambie la arquitectura del transmisor o receptor solo para facilitar la medición. Si radio_rate y bb_rate son diferentes, realice primero la adaptación de tasa y ubique e_EbN0_Sweep después de dicha adaptación.

PROCEDIMIENTO PASO A PASO

1. **Conserve el sistema funcional.** Guarde una copia del flujograma que ya transmite y recibe correctamente con USRP o HackRF One. La nueva copia se utilizará exclusivamente para la medición BER.
2. **Determine la tasa de bits Rb.** Use la tasa real de bits de su sistema. En modulaciones basadas en constelaciones, Rb = Rs × log2(M). En BFSK binaria, Rb = Rs. Coloque este valor en la variable Rb del instrumento.
3. **Determine bb_rate.** Identifique la tasa de muestreo compleja I/Q justo antes de la entrada del receptor digital. Si el SDR trabaja a otra tasa, realice primero la interpolación o decimación requerida.
4. **Integre e_EbN0_Sweep.** Copie el Embedded Python Block **e_EbN0_Sweep** desde BER_SDR_INSTRUMENTACION_BASE.grc. Conecte su entrada a la señal I/Q que normalmente entraba al receptor y conecte su salida compleja al mismo punto de entrada del receptor.
5. **Configure el barrido.** Utilice inicialmente EbN0_min = 0 dB, EbN0_max = 14 dB y EbN0_step = 1 dB. Estos valores producen 15 puntos de evaluación entre 0 y 14 dB.
6. **Obtenga los bits transmitidos de referencia.** Tome una derivación de la secuencia binaria antes de la modulación. En QPSK, 8PSK y QAM debe tomarse antes del agrupamiento de bits en símbolos.
7. **Obtenga los bits recuperados.** La segunda secuencia debe provenir de la salida final del receptor, después de filtrado, sincronización, decisión y demapeo. Debe ser una secuencia de bytes 0/1.

8. Integre e_BER_Curve. Conecte los bits Tx a la entrada 0 y los bits Rx a la entrada 1. Los parámetros de barrido deben ser exactamente los mismos que en **e_EbN0_Sweep**.

9. Alinee Tx y Rx. Ajuste delay_bits en una condición limpia hasta obtener BER aproximadamente igual a cero. Si no se logra, revise retardo, mapeo/demapeo, sincronización o decisión antes de continuar.

10. Configure la cantidad de bits. Use inicialmente bits_por_punto = 100000 y bits_guardia = 1000. Los bits de guarda permiten estabilizar filtros y sincronizadores después de cada cambio de E_b/N_0.

11. Conecte la gráfica. Conecte la salida log10(BER) de e_BER_Curve al QT GUI Vector Sink suministrado. El eje X representa E_b/N_0 [dB] y el eje Y representa log10(BER).

12. Ejecute el sistema completo. El barrido recorre automáticamente los diferentes valores de E_b/N_0 y el medidor acumula errores para cada punto. La curva se irá formando progresivamente sin necesidad de cambiar manualmente el ruido.

13. Permita que la curva se estabilice. El barrido vuelve a comenzar automáticamente y sigue acumulando información. Si en un punto no se observan errores, no debe afirmarse que BER = 0; significa que no se detectaron errores dentro de la cantidad de bits procesados.

14. Conserve el archivo de resultados. Una vez estabilizada la curva, verifique que se haya generado el archivo curva_BER.csv y súbalo a la carpeta correspondiente del proyecto en GitHub. No modifique manualmente sus valores, ya que este archivo constituirá la evidencia numérica utilizada para analizar el desempeño del sistema.

15. Revise la coherencia. El comportamiento esperado es que la BER disminuya al aumentar E_b/N_0. Si la curva no presenta esta tendencia, revise Rb, bb_rate, delay_bits, sincronización, saturación, ruido adicional y la correcta recuperación de bits.

PARÁMETROS PRINCIPALES

Parámetro	Valor inicial sugerido	Interpretación
EbN0_min	0 dB	Inicio del barrido.
EbN0_max	14 dB	Fin del barrido.
EbN0_step	1 dB	Separación entre puntos consecutivos.
Rb	Depende del proyecto	Tasa real de bits del sistema.
Bb_rate	Depende del proyecto	Tasa I/Q en la entrada del receptor.
Bits_por_punto	100000	Cantidad de bits evaluados por punto en cada barrido.
Bits_guardia	1000	Bits ignorados después de cambiar Eb/N0.
Delay_bits	Ajustar	Compensa el retardo total del receptor.
Update_every	1000	Frecuencia de actualización de la curva.

Interpretación de la gráfica

La gráfica utiliza:

x = Eb/N0 [dB]

y = log10(BER)

Por ejemplo:

log10(BER) = -2 ⇒ BER = 10^{-2}

log10(BER) = -3 ⇒ BER = 10^{-3}

INTEGRACIÓN CON HACKRF ONE Y USRP

Con HackRF One:

Osmocom Source → adaptación de tasa → e_EbN0_Sweep → receptor

Con USRP:

UHD: USRP Source → adaptación de tasa → e_EbN0_Sweep → receptor

El instrumento es independiente del radio porque trabaja con muestras complejas I/Q y con bits recuperados. Puede utilizarse con BPSK, QPSK, 8PSK, 16QAM y BFSK siempre que el receptor entregue una secuencia binaria correctamente alineada.

TX Y RX EN COMPUTADORES DIFERENTES

Si el transmisor y el receptor se ejecutan en computadores diferentes, ambos deben conocer la misma secuencia de prueba. Se recomienda utilizar una secuencia conocida y repetible. El receptor debe disponer de esa misma referencia y alinearla con los bits recuperados mediante delay_bits. No utilice secuencias aleatorias independientes en Tx y Rx, porque no sería posible calcular una BER válida.

¿QUÉ SE ESTÁ MIDIENDO?

La señal atraviesa el transmisor digital, el SDR transmisor, el canal físico y el SDR receptor. Después de la recepción, e_EbN0_Sweep agrega una cantidad conocida y controlada de AWGN antes de que la señal ingrese al receptor digital. Por tanto, la curva obtenida combina las imperfecciones reales del enlace SDR con una degradación AWGN adicional controlada.

La primera caracterización debe realizarse con un enlace físico estable y preferiblemente conducido por cable con la atenuación adecuada. Posteriormente puede repetirse el ensayo por aire para analizar las diferencias.

EVIDENCIAS Y ENTREGABLES

- Flujograma completo mostrando claramente dónde se integraron e_EbN0_Sweep y e_BER_Curve.
- Valores utilizados para Rb, bb_rate, EbN0_min, EbN0_max, EbN0_step, bits_por_punto, bits_guardia y delay_bits.
- Explicación del procedimiento empleado para alinear bits Tx y bits Rx.
- Curva BER vs. Eb/N0 obtenida con el sistema implementado.
- Archivo curva_BER.csv generado automáticamente por el sistema de medición. Este archivo deberá subirse a la carpeta del proyecto en GitHub junto con los demás archivos del flujograma. Los datos contenidos en el CSV serán utilizados para evaluar y comparar cuantitativamente el desempeño obtenido por el sistema.
- Captura o datos que demuestren que, en una condición limpia, la BER es aproximadamente cero.
- Análisis técnico de la forma de la curva y de cualquier comportamiento inesperado observado.
- Conclusión sobre el rendimiento del receptor bajo las condiciones evaluadas.

RESULTADO ESPERADO

Al finalizar la actividad, cada equipo debe demostrar que su sistema de comunicaciones SDR puede ser evaluado cuantitativamente mediante una curva BER vs E_b/N_0. No basta con presentar la gráfica: el equipo debe explicar cómo integró el instrumento, cómo verificó la alineación Tx/Rx, qué parámetros utilizó y por qué considera válida la medición.